

Tasarım Desenleri

FinWallet tasarım desenlerini isim kullanmak için değil, belirli finansal problemlere çözüm oldukları yerde kullanır. Gereksiz abstraction veya interface katmanı eklenmemelidir.

DDD-lite

Entity, Value Object, Aggregate ve invariant kavramları finansal modelin sınırlarını açıklamak için kullanılır. Tam event-sourcing veya ağır DDD altyapısı yoktur.

Adapter + Anti-Corruption Layer

IBankProvider, IExternalFraudProvider ve ICommunicationGateway gibi Application portları provider-specific DTO/enumlari Domain'den ayırır. FakeBank/FakeFraud detayları Infrastructure içinde kalır.

Application Orchestrator

Registration, bank-account opening ve transfer gibi use-case'ler handler'lar tarafından sıralı şekilde koordine edilir. Controller business workflow taşımaz.

Chain of Responsibility / Rule Engine

Internal fraud değerlendirmesi bağımsız InternalFraudRule kurallarından oluşur. Yeni bir fraud kuralı mevcut handler'ı büyük ölçüde değiştirmeden eklenebilir.

Policy Pattern

Internal ve external fraud kararlarının birleşimi FraudDecisionPolicy gibi policy nesneleriyle açıkça tanımlanır.

Explicit Unit of Work / SQL Transaction Boundary

Wallet transfer için generic UnitOfWork abstraction yerine SqlWalletTransferPostingStore transaction sınırını açıkça sahiplenir. Locking ve commit sırası business correctness'in parçasıdır.

Idempotent Command

Para değiştiren request client tarafından Idempotency-Key taşır. MSSQL'deki durable record aynı key/same payload replay'ini güvenli yapar; same key/different payload conflict olur.

Optimistic / Compare-and-Set Concurrency

BankAccount gibi lifecycle state update'lerinde expected status + timestamp ile CAS kullanılır. Wallet transfer gibi yoğun para hareketinde gerekli yerde explicit locking/Serializable uygulanır.

Double-Entry Ledger

Her finansal etki Debit/Credit toplamı eşit bir journal ile temsil edilir. Correction geçmiş kaydı değiştirmek yerine reversal/compensation ile yapılır.

State Machine

Authentication/session, BankAccount ve FinancialTransaction lifecycle'ları yalnız izin verilen state geçişleri üzerinden ilerler.

Cache-Aside / TTL

Redis yalnız transient state için TTL tabanlı kullanılır. Financial source of truth değildir.

Timeout / Safe Retry

Provider çağrılarında timeout vardır. Retry yalnız operasyon idempotency açısından güvenliyse uygulanmalıdır; arbitrary financial POST otomatik retry edilmez.

Transactional Outbox / Inbox

Mimari olarak planlanmıştır fakat henüz production flow içinde uygulanmış değildir. Post-commit notification ve duplicate callback işleme için sıradaki desenlerden biridir.

Saga / Compensation

Uzun external-bank workflow'larında gerektiğinde kullanılacaktır. Tek DB içindeki wallet transfer için saga kullanılmaz; tek MSSQL transaction daha doğru ve daha basittir.

Reconciliation

Ledger/balance/provider statement uyumsuzlukları sessizce düzeltilmez; issue olarak tespit edilip izlenmelidir. Reconciliation altyapısı planlanmıştır.

CQRS-lite

Command/query kod organizasyonu ayrıştırılabilir; ayrı read database veya event-sourcing zorunlu değildir.